Practical DevOps

Lesson IaC. Terrafrom Intro

softserve

TERRAFORM

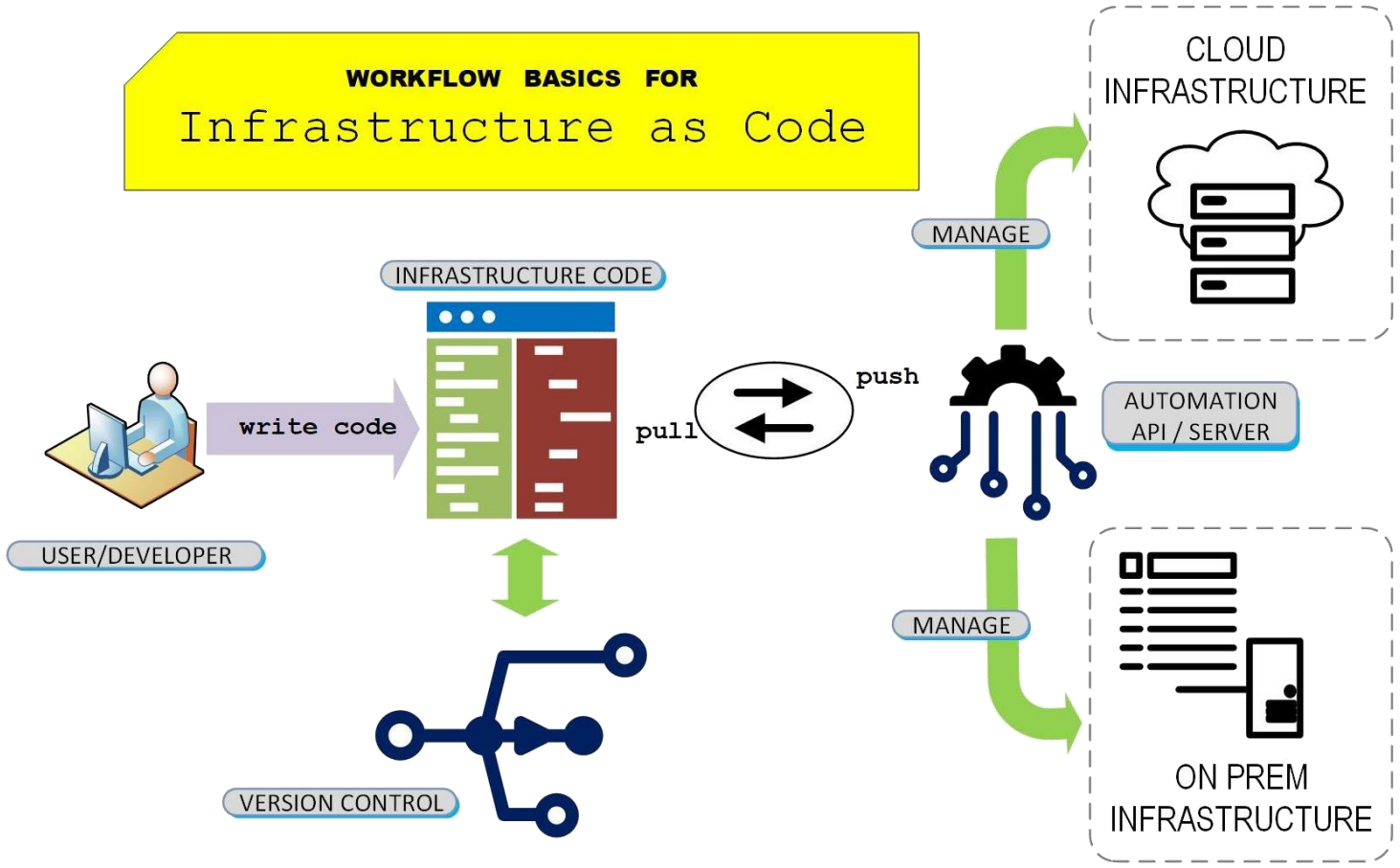
softserve

AGENDA

- IaC Definition
- Pre-History
- IaC Types
  - Configuration Management
  - Server Templating
  - Provisioning Tools
- HashiCorp Terraform
  - Why Terraform?
  - HashiCorp Configuration Language (HCL)
  - Core Concepts
  - Lifecycle (Stages)

softserve

IaC



- **Infrastructure as Code (IaC)** is an IT infrastructure management process that applies best practices from DevOps software development to the management of cloud infrastructure resources.

softserve

Pre-History


BASH
THE BOURNE-AGAIN SHELL
python™


HashiCorp
Vagrant
docker
HashiCorp
Terraform
ANSIBLE
HashiCorp
Packer

IaC

- IaC is a form of **configuration management** that codifies an organization's *infrastructure resources into text files*. These infrastructure files are then committed to a version control system like Git. The version control repository enables feature branch and pull request workflows, which are foundational dependencies of CI/CD.

- **Configuration Management**
  - Ansible
  - Puppet
  - Chef
  - ...
- **Server Templating**
  - Packer (HashiCorp)
  - Vagrant (HashiCorp)
  - Docker
- **Provisioning Tools**
  - Terraform (HashiCorp)
  - AWS CloudFormation
  - Google Cloud Deployment Manager
  - Microsoft Azure Automation

softserve

Configuration Management

- The main purpose:
- Install and manage software on **existing** infrastructure
- Maintains Standard Structure
- Version Control
- **Idempotent**

softserve

Server Templating

HashiCorp Vagrant

HashiCorp Packer

- Pre-Installed Applications and Dependencies
- Virtual Machines (boxes) or containers (images)
- Immutable Infrastructure

softserve

Provisioning Tools

HashiCorp Terraform

- Deploy Immutable Infrastructure Resources
- Virtual Machines, Networks, Storage, Databases, etc.
- Multiple Providers

softserve

HashiCorp

Terraform

softserve

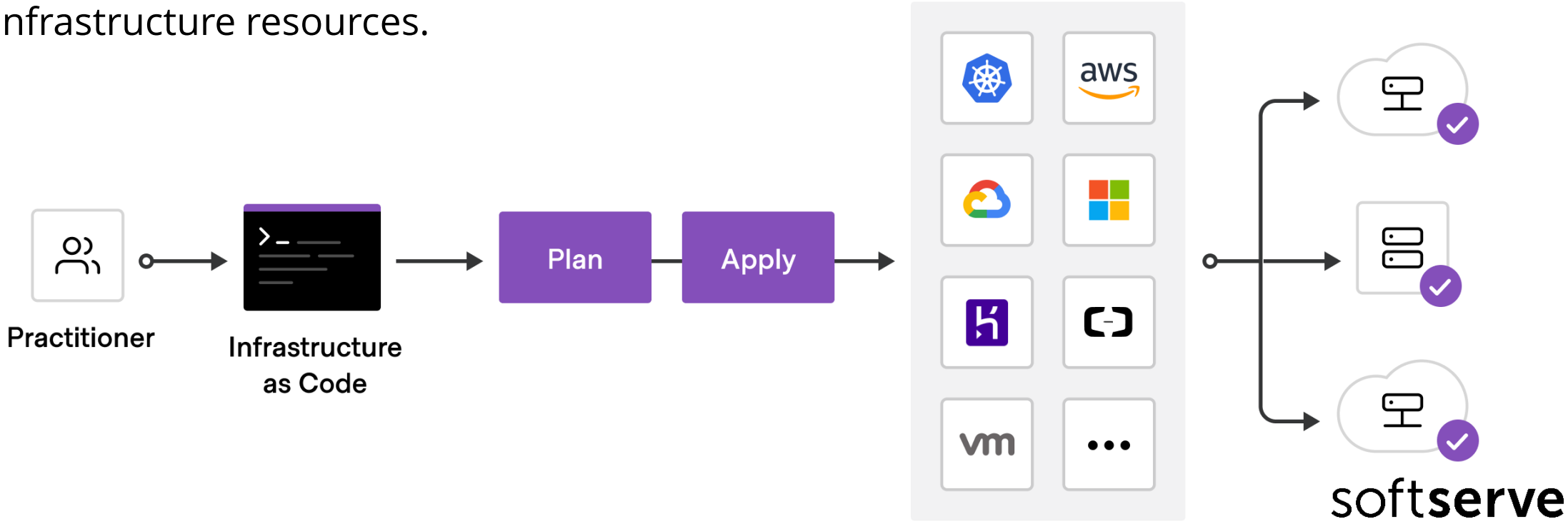
Why Terraform?

HashiCorp Terraform

- Free and Open-Source Tool (written on Go Lang)
- Fast Installation just a single binary package
- Multiplatform (Multi providers)
- Goals
  - **Unify** the view of resources
  - **Support** the modern data center (IaaS, PaaS, SaaS)
  - Expose a way for individuals and teams to **safely and predictably change infrastructure**
  - Provide a workflow that is **technology agnostic**
  - Manage anything with an **API**

IaC & Terraform

- **Infrastructure as Code (IaC)** is an IT infrastructure management process that applies best practices from DevOps software development to the management of cloud infrastructure resources.



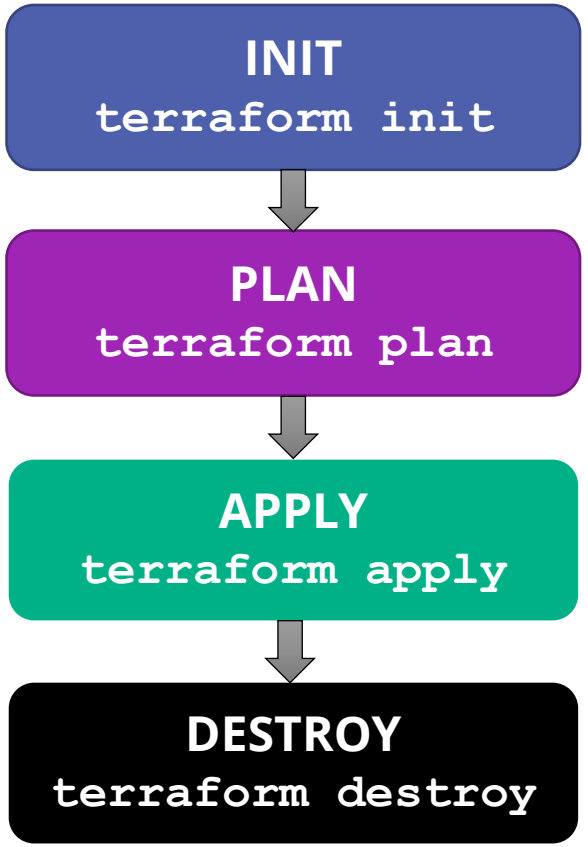
Terraform CORE Concepts

- **Variables:** Also used as input-variables, it is key-value pair used by Terraform modules to allow customization.
- **Provider:** It is a plugin to interact with APIs of service and access its related resources.
- **State:** It consists of cached information about the infrastructure managed by Terraform and the related configurations.
- **Resources:** It refers to a block of one or more infrastructure objects (compute instances, virtual networks, etc.), which are used in configuring and managing the infrastructure.
- **Output Values:** These are return values of a terraform module that can be used by other configurations.

softserve

Stages (Lifecycle)

- **Init** – Initialize a working directory containing Terraform configuration files. This is the first command that should be run after writing a new Terraform configuration or cloning an existing one from version control
- **Plan** – creates an execution plan. By default, creating a plan consists of:
  - Reading the **current state** of any already-existing remote objects to make sure that the Terraform state is up-to-date
  - **Comparing** the current configuration to the prior state and noting any differences.
  - **Proposing** a set of change actions that should, if applied, make the remote objects match the configuration.
- **Apply** – execution current plan. Create infrastructure.
- **Destroy** – destroyed infrastructure , what applied to plan.



softserve

State

- Terraform must store state about your managed infrastructure and configuration.
- This state is used by Terraform to map real world resources to your configuration, keep track of metadata, and to improve performance for large infrastructures.
- This state is stored by default in a local file named "terraform.tfstate", but it can also be stored remotely, which works better in a team environment.
- The primary purpose of Terraform state is to store **bindings** between objects in a remote system and resource instances declared in your configuration.

Resources

- Any object of real world ☺ like:
  - Local File
  - Provider Resources (AWS: EC2, S3, VPC, DynamoDB, RDS, ...)
  - Provider Resources (GCP: CE, ...)

HashiCorp Configuration Language HCL

- **HashiCorp Configuration Language (HCL)** is a unique configuration language. It was designed to be used with *HashiCorp tools*, notably **Terraform**, but HCL has expanded as a more general configuration language. It's visually like JSON with additional data structures and capabilities built-in.
- HCL consists of three sub-languages:
  - Structural
  - Expression
  - Templates
- The main purpose of the Terraform language is declaring **resources**, which represent infrastructure objects.
- All other language features exist only to make the *definition of resources* more flexible and convenient.

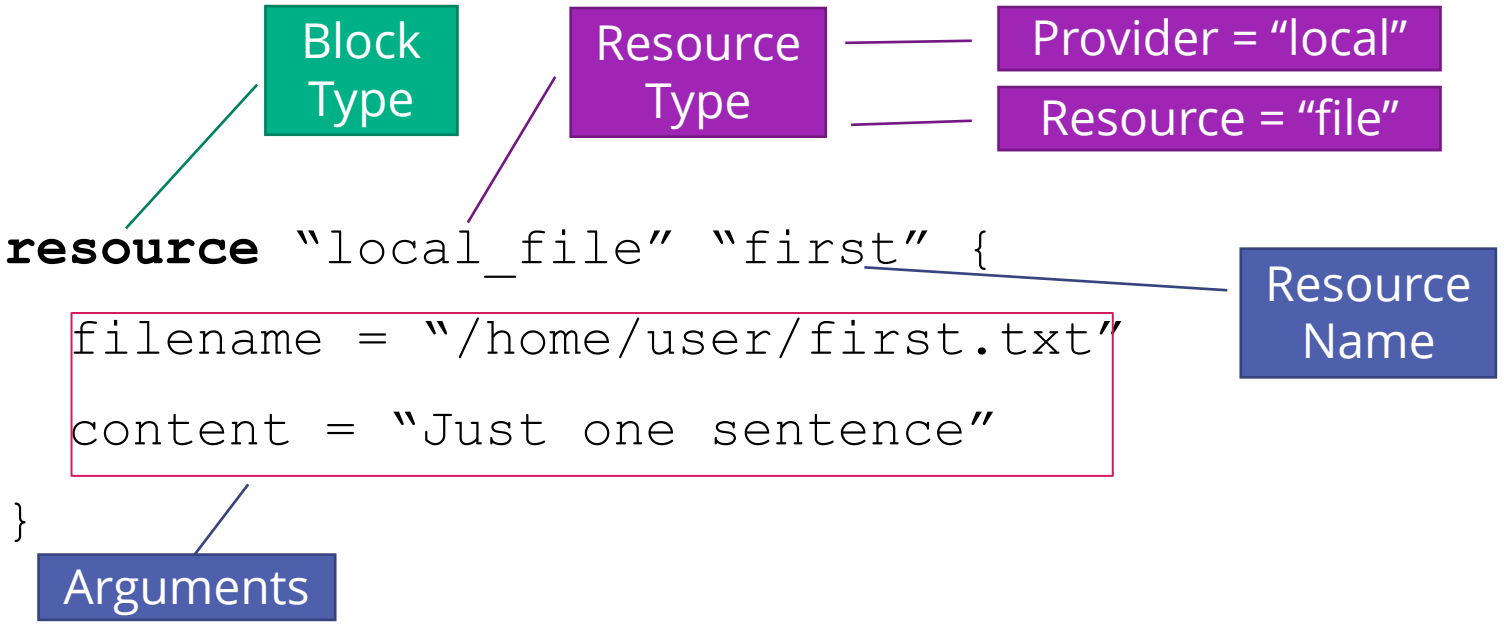
HCL (Terraform)

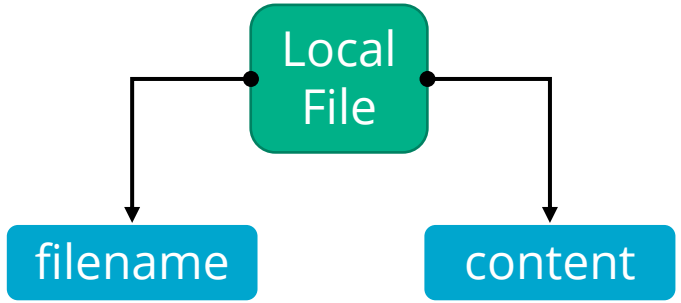
- **Blocks** are containers for other content and usually represent the configuration of object, like a resource. Blocks have a block type, can have zero or more labels, and have a body that contains any number of arguments and nested blocks. Most of Terraform's features are controlled by top-level blocks in a configuration file.
- **Arguments** assign a value to a name. They appear within blocks.
- **Expressions** represent a value, either literally or by referencing and combining other values. They appear as values for arguments, or within other expressions.

```
<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {
  # Block body
  <IDENTIFIER> = <EXPRESSION> # Argument
}
```

softserve

HCL First Example

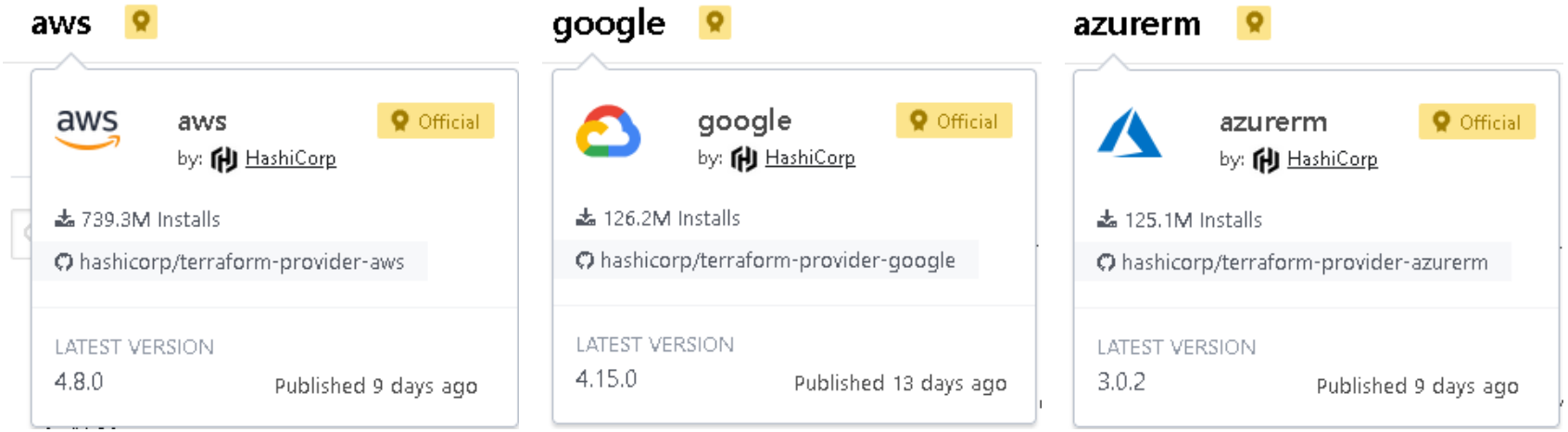




softserve

CLOUD RESOURCES

- AWS – https://registry.terraform.io/providers/hashicorp/aws/latest/docs
- GCP – https://registry.terraform.io/providers/hashicorp/google/latest/docs
- Azure – https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs



softserve

main.tf U

C: > DOCS_VOLUME > SS_ITA_WORK > 2022 > DevOps-Spring-School-2022 > main.tf

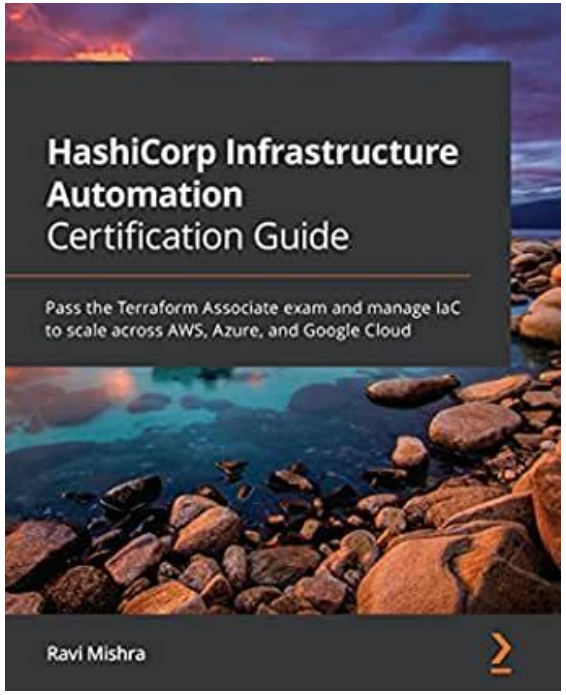
```
terraform {
  required_providers {
    google = {
      source = "hashicorp/google"
      version = "3.5.0"
    }
  }
}

provider "google" {
  credentials = file("my_gcp.json")

  project = "project_vasis"
  region  = "europe-north1"
  zone    = "europe-north1-a"
}

resource "google_compute_instance" "vm_instance" {
  name         = "moodle"
  machine_type = "e2-small"

  boot_disk {
    initialize_params {
      image = "ubuntu-2004-focal-v20210720"
    }
  }
}
```

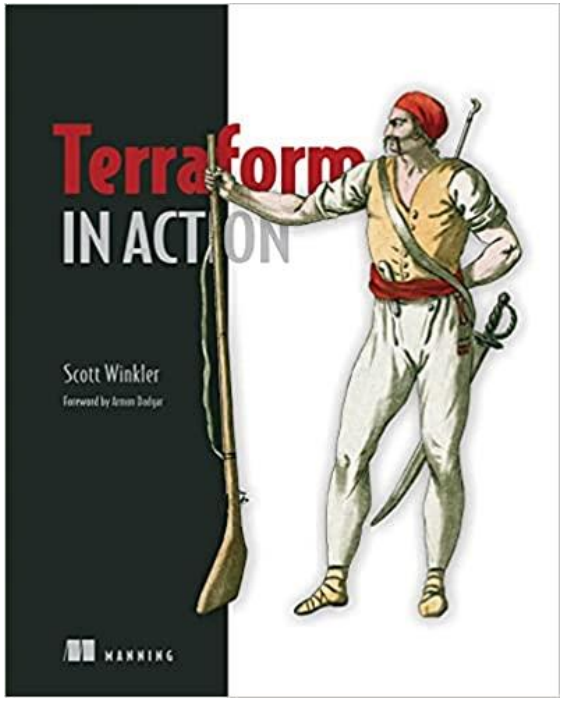
GCP creating virtual machine instance using HashiCorp Terraform

softserve

REFERENCES & SOURCES

- HashiCorp Learn Terraform Tutorials https://learn.hashicorp.com/terraform
- HashiCorp Infrastructure Automation Certification Guide by Ravi Mishra
- Terraform in action by Scott Winkler





softserve

To be continued ...

softserve


NO
THE
FUTURE
softserve